

잘 짤 앱을 받아 '내 앱'으로

할 일 체크리스트 PWA · 받아서 개조 · 강사 진행 대본 (정금구)

사용법

한 페이지 = 한 슬라이드. ★ KEY POINT(보라) = 반드시 전달할 한 문장 · 이렇게 말하세요(검은 박스) = 실제 멘트 · 강사 팁(노랑) · 참고(회색) · → 넘어가며.

용어 한 줄 (강사용)

PWA 웹인데 폰에 설치·오프라인 되는 앱 · **레이어** 역할별로 나눈 파일 겹(데이터 store·동작 actions·화면 render·연결 app) · 순수 함수 입력만 받아 출력만 내는 함수(actions) · **매니페스트** 이름·아이콘·색 · **서비스워커** 캐시→오프라인(코드 바꾸면 버전↑) · **localStorage** 폰에 데이터 저장 · **HTTPS/Vercel** 자물쇠 주소/무료 배포처

구분	시간	슬라이드
오프닝+복습	6분	1~3
Part 1 — PWA란? + 5요소	10분	4~5
Part 2 — 스타터 받기·실행	6분	6
Part 3 — 구조 투어	18분	7~9
Part 4 — 디자인 개조	14분	10~11
Part 5 — 기능 개조	15분	12~13
Part 6 — 캐시·배포	19분	14~17
Part 7 — 폰 설치·공유	10분	18~19
마무리	8분	20~22

표지 — "폰에 깔리는 앱"

1분

★ KEY POINT

오늘은 잘 짜인 앱을 받아 내 입맛대로 고쳐, 내 폰에 깎는다.

이렇게 말하세요

6차에선 맨손으로 만들었죠. 오늘 7차는 다릅니다 — 잘 짜인 할 일 앱을 받아서, 구조를 이해하고, 디자인이랑 기능을 내 것으로 고친 다음, 폰에 깔고 친구한테 보낼 거예요.

강사 팁

6차 수강 전제. Claude Code 기본은 빠르게, '구조 읽기 + 개조 + PWA 배포'에 집중.

→ 넘어가며: "지금까지 어디까지 왔는지부터."

6차는 맨손으로, 7차는 개조로

3분

★ KEY POINT

4차 미리보기 → 6차 맨손 빌드 → 7차 잘 짠 앱 개조·배포. 한 단계 위로.

이렇게 말하세요

4차는 브라우저 미리보기, 6차는 내 컴퓨터에서 맨손으로 만들었죠. 오늘은 잘 짜인 걸 받아 이해하고 고치는 거예요. 만드는 게 아니라 '읽고 고치는' 게 핵심입니다 — 사실 이게 실무에서 더 많이 하는 일이에요.

→ 넘어가며: "그래서 오늘 세 가지가 달라집니다."

읽고 · 고치고 · 배포한다

2분

★ KEY POINT

구조를 읽는다 → 내 것으로 개조한다(디자인+기능) → 배포·설치한다.

이렇게 말하세요

셋이에요. 하나, 잘 나뉜 구조를 읽으며 "왜 이렇게 짰나"를 배웁니다. 둘, 디자인과 기능을 내 입맛대로 고쳐요. 셋, 배포해서 폰에 깔고 링크로 공유. 끝나면 '받은 앱'이 아니라 '내가 만든 앱'이 됩니다.

→ 넘어가며: "**PWA**가 원지부터."

앱 = 웹페이지 + 다섯 가지

5분

★ KEY POINT

앱스토어·네이티브 없이, 웹에 5요소를 더하면 폰에 깔리는 앱이 된다.

이렇게 말하세요

PWA는 웹으로 만들었는데 앱처럼 설치·오프라인 되는 거예요. 네이티브 앱은 전용 언어에 앱스토어 심사까지 필요하지만, PWA는 우리가 아는 웹에 다섯 가지만 더하면 됩니다. 여러분 폰의 앱 중 일부도 사실 PWA예요.

→ 넘어가며: "그 다섯 — 화·정·오·저·배."

PWA 5요소 — 화·정·오·저·배

5분

★ KEY POINT

화면·정보·오프라인·저장·배포. 오늘 '받은 앱에서 이 다섯을 찾는다'.

이렇게 말하세요

화는 화면, 정은 정보(이름·아이콘=매니페스트), 오는 오프라인(서비스워커), 저는 저장(store.js의 localStorage), 배는 배포. 오늘은 이걸 직접 만드는 게 아니라, 받은 앱에서 '어느 파일이 이 요소인지' 찾을 거예요. 그리고 숨은 전제 — 설치랑 오프라인은 HTTPS에서만 됩니다.

강사 팁

칠판에 '화·정·오·저·배' + '레이어 4겹' 적어두고 강의 내내 가리키기.

→ 넘어가며: "자, 그 앱을 받아 실행해 봅시다."

이미 작동하는 앱을 받았다

6분

★ KEY POINT

압축 풀고 **node server.js** → **localhost**. 벌써 추가·체크·삭제가 된다.

이렇게 말하세요

나눠드린 압축을 풀고, 그 폴더에서 Claude Code에 "로컬 서버로 띄워줘" 하세요. localhost를 열면 — 이미 작동하는 할 일 앱이 떠요. 추가도 되고 체크도 되죠. 이제 이걸 뜯어볼 겁니다.

강사 팁

모두 같은 스타터에서 출발 → 라이브 변수 최소. 모바일 뷰로 폰처럼도 보여주기.

→ 넘어가며: "이게 어떻게 만들어졌는지 — 구조 투어."

레이어 4겹 — 한 파일 = 한 책임

7분

★ KEY POINT

데이터(store)·동작(actions)·화면(render)·연결(app). 역할별로 나뉘어 '읽힌다'.

이렇게 말하세요

잘 짜인 앱은 파일이 역할별로 나뉘어요. store는 저장만, actions는 계산만(목록을 받아 새 목록을 돌려줌), render는 그리기만, app은 이걸 잇기만. Claude Code에 "이 파일들이 뭐 하는지 쉽게 설명해줘" 하면 한눈에 정리해줍니다.

강사 팁

specs/레이어_구조.md를 같이 열어 보여주기. @actions.js로 한 파일씩 설명 요청도 시연.

→ 넘어가며: "그럼 데이터가 어떻게 흐르나?"

입력이 레이어를 타고 흐른다

6분

★ KEY POINT

입력 → **app** → **actions(계산)** → **store(저장)** → **render(화면)**. 모든 변경은 **update()** 한 곳.

이렇게 말하세요

할 일을 입력하면, app이 받아서 actions에 넘기고, actions가 새 목록을 계산하고, store가 저장하고, render가 화면을 다시 그려요. 앞 단계의 결과가 다음 단계의 재료. 그리고 저장과 화면은 항상 update()라는 한 곳을 거쳐서, 흐름이 안 꼬여요.

참고

2차 'I/O 계약', 6차 '레이어'가 이 앱에 그대로 들어 있다 — 추상이 아니라 실물.

→ 넘어가며: "그래서 — 구조를 읽으면 고칠 곳이 보입니다."

구조를 읽으면 고칠 곳이 보인다

5분

★ KEY POINT

기능 추가는 **actions**부터, 모양 바꾸기는 **render·css**부터. 좋은 구조 = 고치기 쉬운 구조.

이렇게 말하세요

이게 오늘의 핵심이에요. 한 파일이 한 가지만 하니까, 뭘 고칠 때 어디를 건드릴지 바로 보입니다. 새 기능은 actions에 계산을 추가하고, 색이나 모양은 css.render를 손대면 돼요. 잘 짠 코드를 '읽는 것'만으로 이걸 배웁니다.

강사 팁

"여러분은 오늘 코드를 '구경'하는 게 아니라 '읽고 고치는' 겁니다" — 학습 프레이밍 강조.

→ 넘어가며: "가벼운 것부터 고쳐봅시다 — 디자인."

가볍게 — 내 취향으로

7분

★ KEY POINT

앱 이름·테마색·테마·둥글기 — 즉시 결과가 보이는 것부터. 작은 성공이 쌓인다.

이렇게 말하세요

먼저 쉬운 것. "앱 이름을 바꿔줘", "테마색을 내 색으로", "밝은 테마로". 새로고침하면 바로 바뀌어요. 색·이름은 본인 취향대로 자유롭게 — 벌써 내 앱 같죠?

강사 팁

각자 다른 색/이름으로 바꾸게 하면 '내 것' 느낌이 확 산다. 30초 미션처럼.

→ 넘어가며: "아이콘도 내 이미지로 바꿔봅시다."

내 이미지를 홈 화면 아이콘으로

7분

★ KEY POINT

Claude Code에 이미지 '파일 경로'를 알려주면, 가져와 192·512 아이콘으로 만들어 연결한다.

이렇게 말하세요

미리 준비한 이미지(직접 그렸든 AI로 만들었든)를 쓸 거예요. 그 파일을 Git Bash 창에 끌어다 놓으면 경로가 찍혀요. 그 경로를 Claude Code에 주면서 "이 이미지를 192·512 아이콘으로 만들어 연결해줘" 하면 끝. 'Claude Code에 파일 경로를 알려주면 그 파일을 가져다 쓴다'는 것도 같이 배우는 거죠.

강사 팁

경로 인식 안 되면 이미지를 프로젝트 폴더에 넣고 파일명으로. Application→Manifest에서 확인.

→ 넘어가며: "이제 한 단계 위 — 기능을 더해봅시다."

어느 레이어를 고치나

8분

★ KEY POINT

"어느 파일(레이어)을 고치는지"를 같이 말하면 결과가 정확하고, 구조를 이해했다는 신호.

이렇게 말하세요

기능을 추가할 땐 순서가 있어요. 데이터 계산은 actions에, 연결은 app에, 보여주는 render에. 그래서 Claude Code에 시킬 때 "actions에 함수 만들고, render에 표시해줘"처럼 어느 레이어인지 같이 말하면 정확하게 붙어요. 보여주는 방식만 바꾸는 건(통계·정렬) render만 손대면 되고요.

→ 넘어가며: "하나 골라서 직접 붙여봅시다."

하나만 골라 붙여본다

7분

★ KEY POINT

중요 표시·완료율 통계·완료 숨기기 등 하나 추가 → "이건 내 앱"이 된다.

이렇게 말하세요

예를 들어 "중요 표시(별표) 기능 추가해줘 — actions에 함수, render에 별표, 중요한 건 위로". 구조가 좋아서 정확한 자리에 붙어요. 통계나 완료 숨기기 같은 것도 좋고요. 하나만 붙여도, 이제 받은 앱이 아니라 '내가 만든 앱'입니다.

강사 팁

수강생마다 다른 기능을 고르게 하면 결과물이 다 달라져 성취감↑. 막히면 "actions부터, 그다음 render" 순서로.

→ 넘어가며: "고쳤으니 — 캐시를 갱신해야 해요."

고쳤으면 캐시 버전을 올린다

4분

★ KEY POINT

서비스워커가 옛 파일을 캐시해 뒀서, 버전 안 올리면 옛 화면이 보인다(v2→v3).

이렇게 말하세요

서비스워커가 앱 파일을 저장해두기 때문에, 코드를 바꿨는데 옛날 화면이 보일 때가 있어요. 그럴 땐 sw.js의 캐시 버전을 올리면 됩니다. "캐시 버전 올려줘" 한마디. '이상하면 캐시 버전 올리기' — 이거 하나 외우면 평생 덜 헤매요.

→ 넘어가며: "이제 진짜 폰에 깔려면 — 배포."

진짜 설치하는 배포 후에

4분

★ KEY POINT

설치·오프라인은 **HTTPS**에서만. 로컬은 테스트용 — 배포해야 폰에 깔린다.

이렇게 말하세요

폰에 깔리려면 'HTTPS'라는 자물쇠 달린 안전한 주소여야 해요. 내 컴퓨터 localhost는 테스트용 예외고, 진짜 폰에 깔려면 인터넷에 올린 HTTPS 주소가 필요합니다. 그래서 배포해요.

→ 넘어가며: "가장 쉬운 배포 — **Vercel.**"

Vercel로 진짜 URL

8분

★ KEY POINT

GitHub에 올리고 Vercel 연결 → 자동 HTTPS URL. 정적 앱이라 설정 거의 없음.

이렇게 말하세요

"GitHub에 올려줘" → "Vercel에 배포하는 법 알려줘". 코드를 GitHub에 올리고 Vercel에서 그 저장소를 고르면, 자동으로 HTTPS 주소를 만들어줍니다. 무료고 설정도 거의 없어요. Railway에 익숙하면 server.js로 거기 올려도 됩니다.

강사 팁

계정·인증 변수 큼 → 사전 로그인 안내, 막히면 강사 배포 화면 공유.

→ 넘어가며: "6차에선 이게 안 됐죠? 그 차이."

무엇을 만드느냐가 배포를 정한다

3분

★ KEY POINT

6차 변경기는 내 파일 만져 **URL** 배포 불가였지만, 할 일 앱은 안 만지니 **URL** 배포가 정답.

이렇게 말하세요

6차에서 '내 파일 만지는 도구는 URL 배포로 남이 못 쓴다'고 했죠. 오늘 할 일 앱은 남의 파일을 안 만지고 자기 폰에만 저장하니까 정반대 — URL 배포가 정답입니다. 무엇을 만드느냐가 배포 방식을 정합니다.

→ 넘어가며: "이제 진짜 폰에 깔아봅시다."

내 폰 홈 화면에 깐다

6분

★ KEY POINT

배포 **URL**을 폰에서 열고 "홈 화면에 추가". 안드로이드/아이폰 방식이 다름.

이렇게 말하세요

발급된 주소를 여러분 폰 브라우저에서 여세요. 안드로이드 크롬은 '홈 화면에 추가' 배너/메뉴, 아이폰은 사파리 공유 버튼 눌러 '홈 화면에 추가'. 누르면 홈 화면에 아이콘이 생기고, 열면 주소창 없는 앱 모드로 떠요.

강사 팁

아이폰 설치 UX가 달라 헤맸 → Safari 공유 메뉴 스크린샷 미리 띄워두기.

→ 넘어가며: "그리고 친구한테도."

내가 개조한 앱이 폰에 산다

4분

★ KEY POINT

URL을 단톡방에 공유 → 누구나 설치. 비행기모드로 오프라인까지 확인.

이렇게 말하세요

그 주소를 단톡방에 보내보세요. 받은 친구도 똑같이 깔 수 있어요. "어 이거 네가 만든 거야?" 소리 납니다. 비행기모드로 열어 인터넷 없이도 되는 것까지 보여주면, 받은 게 아니라 '내가 만든 앱'이 완성된 거예요.

→ 넘어가며: "오늘 만진 걸 정리하고 마칩니다."

PWA 5요소 + 레이어 4겹

3분

★ KEY POINT

이 두 지도(5요소·레이어 4겹)면, 다음 앱도 읽고 고쳐 폰에 깔 수 있다.

이렇게 말하세요

오늘 두 가지 지도를 얻었어요. PWA 5요소(화·정·오·저·배)로 '앱이 되려면 뭐가 필요한지', 레이어 4겹(store·actions·render·app)으로 '코드를 어떻게 나누고 어디를 고치는지'. 이 둘이면 다음에 다른 앱을 받아도 읽고 고쳐서 폰에 깔 수 있어요.

→ 넘어가며: "습관 세 가지로 압축."

오늘부터, 이 세 가지

2분

★ KEY POINT

① 고치기 전에 구조부터 읽기 ② 디자인 → 기능 순서 ③ 캐시 버전·HTTPS.

이렇게 말하세요

하나, 고치기 전에 구조부터 — 어느 레이어인지 보기. 둘, 쉬운 디자인부터 고쳐 성공 경험 쌓고 기능으로. 셋, 코드 바꾸면 캐시 버전 올리고, 설치·오프라인은 배포 후 HTTPS에서 확인.

→ 넘어가며: "질문 받겠습니다."

고맙습니다 · Q&A

3분

★ KEY POINT

읽고 · 고치고 · 배포한다 — 이제 어떤 앱이든 내 것으로.

이렇게 말하세요

오늘 잘 짠 앱을 받아 구조를 읽고, 디자인·기능을 내 것으로 고치고, 배포해 폰에 깔았습니다. 핵심은 '구조를 읽으면 고칠 곳이 보인다'예요. 이걸 알면 세상 어떤 코드든 덜 무서워집니다. 질문 받을게요. 고맙습니다.

강사 팁

"다음엔 이 앱에 푸시 알림·카메라 같은 폰 기능을 더해보자"로 다음 강의 예고.

→ 끝.